Etapa 2 – Analisador Sintático (Bison)

Lívia Maria Almeida Silva
Universidade Federal de Lavras – UFLA
Departamento de Ciência da Computação
Lavras, Minas Gerais, Brasil

Mauricio Vicente Sandim
Universidade Federal de Lavras – UFLA
Departamento de Ciência da Computação
Lavras, Minas Gerais, Brasil

1 Introdução

Este relatório apresenta a segunda etapa do trabalho de Compiladores, cujo objetivo é desenvolver, utilizando o *Bison*, o analisador sintático de uma linguagem de programação simplificada. São apresentadas as decisões de projeto, as estratégias utilizadas para a resolução de ambiguidades, o tratamento de erros sintáticos e as dificuldades encontradas durante o desenvolvimento. Por fim, é apresentada a gramática completa em BNF.

2 Decisões de Projeto

2.1 Estrutura da Linguagem

A linguagem foi definida para reconhecer declarações de variáveis, atribuições, expressões aritméticas, relacionais e lógicas, estruturas condicionais, estruturas de repetição e comandos de entrada e saída.

Os *tokens* utilizados pelo analisador sintático, fornecidos pelo analisador léxico, são apresentados a seguir:

```
TIPO ID NUMERO
IF ELSE WHILE PRINT READ
MAIS MENOS MULT DIV MOD ATRIBUICAO
RELOP E OU NAO
PONTO_VIRGULA VIRGULA
ABRE_PARENTESES FECHA_PARENTESES
ABRE_CHAVES FECHA_CHAVES
ERRO_LEX
```

As declarações permitem a definição de múltiplas variáveis do mesmo tipo em uma única instrução, com inicialização opcional. Além disso, a linguagem permite a utilização de comandos aninhados.

As estruturas condicionais e de repetição podem conter comandos isolados ou blocos delimitados por chaves. Também são permitidos encadeamentos `else if`.

2.2 Precedência e Resolução de Ambiguidades

A gramática utiliza declarações de precedência e associatividade para resolver conflitos de análise em expressões aritméticas, relacionais e lógicas.

Em ordem crescente de precedência, os operadores são organizados da seguinte maneira: operador lógico OU, operador lógico E, operadores relacionais, operadores de adição e subtração e, posteriormente, operadores de multiplicação, divisão e módulo.

Os operadores unários de negação lógica e sinal negativo possuem a maior precedência. Para diferenciar o operador de menos unário do operador binário de subtração, foi utilizada a diretiva `%prec`, atribuindo maior precedência à operação unária.

Outra decisão importante foi associar o comando `else` ao `if` correspondente mais próximo que ainda não possua um `else`, evitando interpretações ambíguas em estruturas condicionais aninhadas.

2.3 Tratamento de Erros

Para o tratamento de erros, a gramática utiliza o símbolo especial `error` do *Bison*.

Foram definidos dois pontos principais de sincronização: o ponto e vírgula (`;`) e o fechamento de chaves (`}`). Ao encontrar um erro, o analisador pode descartar parte da entrada até alcançar um desses símbolos e tentar continuar o processamento.

O comando `yyerrok` é utilizado para encerrar o estado de recuperação, permitindo que novos erros sejam identificados.

O *token* `ERRO_LEX` é reservado para sinalizar entradas não reconhecidas pelo analisador léxico.

Durante a análise, os erros léxicos e sintáticos são registrados e reportados com suas respectivas linhas e colunas. Ao término do processamento, o programa verifica a ocorrência desses erros antes de informar que a análise foi concluída com sucesso. Independentemente do resultado, também é realizada a impressão da tabela de símbolos gerada pelo analisador léxico.

3 Resultados e Dificuldades Encontradas

Uma das principais dificuldades encontradas durante o desenvolvimento foi a definição da precedência e da associatividade dos operadores aritméticos, relacionais e lógicos. Para resolver os conflitos de análise, foram utilizadas as diretivas de precedência do *Bison*, estabelecendo a prioridade e a associatividade de cada grupo de operadores.

Outra dificuldade foi o tratamento da ambiguidade *dangling else*. Para evitar diferentes interpretações, foi adotada a associação do `else` ao `if` mais próximo que ainda não possua um `else`.

Também foram encontradas dificuldades na implementação da recuperação de erros sintáticos. Embora o analisador consiga retomar o processamento após encontrar determinados pontos de sincronização, essa estratégia pode ocasionar erros em cascata.

Esse comportamento foi observado quando um erro ocorreu antes da abertura de um bloco delimitado por chaves. Nessa situação, o analisador pôde descartar a abertura do bloco durante a recuperação e retomar o processamento ao encontrar um ponto e vírgula em seu interior. Entretanto, ao alcançar o fechamento de chaves correspondente, esse símbolo foi interpretado como inesperado, gerando uma nova mensagem de erro sintático.

Dessa forma, um único erro na entrada pode ocasionar outras mensagens de erro em trechos posteriores do código. Apesar dessa limitação, o analisador identifica o erro inicial e tenta continuar o processamento, permitindo a detecção de outros problemas sintáticos no arquivo.

4 Gramática em BNF

```
<programa> ::= <lista_itens>

<lista_itens> ::= ε
                | <lista_itens> <item>
                | <lista_itens> error PONTO_VIRGULA

<item> ::= <declaracao>
         | <comando>

<declaracao> ::= TIPO <lista_declaradores> PONTO_VIRGULA

<lista_declaradores> ::= <declarador>
                       | <lista_declaradores> VIRGULA <declarador>

<declarador> ::= ID
               | ID ATRIBUICAO <expressao>

<comando> ::= <atribuicao>
            | <condicional>
            | <repeticao>
            | <impressao>
            | <leitura>
            | <bloco>

<atribuicao> ::= ID ATRIBUICAO <expressao> PONTO_VIRGULA

<condicional> ::= IF ABRE_PARENTESES <expressao> FECHA_PARENTESES <item>
                | IF ABRE_PARENTESES <expressao> FECHA_PARENTESES <item> ELSE <item>

<repeticao> ::= WHILE ABRE_PARENTESES <expressao> FECHA_PARENTESES <item>

<impressao> ::= PRINT ABRE_PARENTESES <expressao> FECHA_PARENTESES PONTO_VIRGULA

<leitura> ::= READ ABRE_PARENTESES ID FECHA_PARENTESES PONTO_VIRGULA

<bloco> ::= ABRE_CHAVES <lista_itens> FECHA_CHAVES
          | ABRE_CHAVES <lista_itens> error FECHA_CHAVES

<expressao> ::= <expressao> OU <expressao>
              | <expressao> E <expressao>
              | <expressao> RELOP <expressao>
              | <expressao> MAIS <expressao>
              | <expressao> MENOS <expressao>
              | <expressao> MULT <expressao>
              | <expressao> DIV <expressao>
              | <expressao> MOD <expressao>
              | NAO <expressao>
              | MENOS <expressao>
              | ABRE_PARENTESES <expressao> FECHA_PARENTESES
              | ID
              | NUMERO
```